BUSINESS REQUIREMENTS DOCUMENT

Multi-Agent Knowledge Assistant

Enterprise AI Knowledge & Research Platform

Version 2.1 (supersedes v2.0 of 30 October 2026 and v1.0 of 29 October 2026) | **Date:** 30 October 2026

Technology: Python 3.12, FastAPI, LangGraph, SQLAlchemy + Alembic, PyMuPDF, pandas, PostgreSQL 16, Redis 7, Qdrant / ChromaDB, OpenAI / Gemini (optional), React + TypeScript, Prometheus, OpenTelemetry

Purpose	Define goals, users, scope, requirements, architecture, security, evaluation and acceptance criteria of the platform, and record - requirement by requirement - what is implemented, how it was verified and what is not.
Status labels	**Implemented, tested**: implemented and covered by automated tests that were executed. **Implemented, partly tested**: implemented, but a dependency (live LLM API, model download, container runtime, external server) was not available in the verification environment, so only the stated part was tested. **Pending**: not implemented.
What changed in v2.1	Alembic migrations and a PostgreSQL document registry (versions, ACL); PyMuPDF engine, XLS/HTML, duplicate detection and versioning; classification, timeline, page previews; data-analyst toolkit with a sandboxed SQL engine and charts; Comparison Agent, contradiction detection, Self-RAG, citation validation; parallel agents, timeouts, metrics, tracing, retention scheduler; reports in PDF/DOCX/XLSX; user-controlled memory; refresh tokens and roles user/manager/admin; full 14-screen UI; extended evaluation and performance measurements.

Revision history

Version	Date	Change
1.0	29 Oct 2026	Initial BRD: multi-format ingestion, hybrid search, exact counts, table analysis, LangGraph routing.
2.0	30 Oct 2026	Multi-tenant auth and RBAC, PostgreSQL/Qdrant/Redis, background workers, advanced RAG, 8-agent graph with verification, evaluation harness, React dashboard, Docker/CI.
2.1	30 Oct 2026	Enterprise research platform upgrade (this document). Requirements are traced to code and tests in docs/REQUIREMENTS_TRACE.md.

1. Executive Summary

The platform lets people upload large, mixed-format document collections (or point at approved web pages), ask questions in natural language and receive answers backed by verifiable evidence - file, version, page, sheet and row, section or URL. Version 2.1 adds what an enterprise research workflow needs on top of v2.0: a document registry with versions and sharing, deeper ingestion (PyMuPDF, XLS, HTML, duplicates, classification, timelines), a data-analyst toolkit with a sandboxed SQL engine, document comparison with change and contradiction detection, a Self-RAG loop and citation validation, parallel multi-agent answers, report generation in PDF/DOCX/XLSX, user-controlled memory, retention and observability.

Design principles

- **Evidence or abstention.** Answers cite their sources; every citation is re-checked against the stored text (page/row exists, excerpt is there). Insufficient evidence yields an explicit 'could not verify'.
- **Code computes, models explain.** Counts and figures come from deterministic code (pandas, a read-only SQL sandbox) over the complete data. No generated code or SQL is ever executed outside the sandbox's authorizer.
- **Documents are data, never instructions.** Retrieved text is fenced and scanned; the tool set is closed; HyDE text is never evidence; web evidence is kept apart from document evidence.
- **Isolation by construction.** Tenant, workspace and document ACL are enforced below the API layer; hidden documents are indistinguishable from missing ones.
- **Measure, then claim.** Quality, latency, cost, citation accuracy, agent success and tool failures are measured; limits are documented with numbers.
- **Honest status.** Implemented-and-tested, partly tested and pending are labelled per requirement.

Verification summary (executed 30 Oct 2026 in the build environment)

Suite	Result	Scope
Backend (pytest)	337 passed, 0 failed (0 skipped)	ingestion incl. a 1,000-page PDF, OCR, migrations, registry/versions/ACL, auth/refresh/roles, RAG, agents, analytics + SQL sandbox, comparison, Self-RAG, reports, memory, metrics/tracing, scheduler, evaluation; real PostgreSQL 16 and Redis 7
Frontend (Vitest)	38 passed	all screens with mocked HTTP: dashboard, library, exact search, comparison, analyst + charts, reports, history, settings, preview, SSE, token refresh, XSS escaping
End-to-end (Playwright)	18 / 18 steps	real server + built app + headless Chromium: sign-up, silent refresh, upload, duplicate and version flow, cited answers, exact search with PDF page preview, parallel answer, incognito, comparison, analyst, PDF report download, theme persistence, admin, role gating
Production-shaped smoke	12 / 12 checks	separate API and worker processes, PostgreSQL (data + checkpoints), Redis, CLI tenant bootstrap, production-mode validation, Alembic migrations at start
Static checks	ruff clean; tsc --strict clean; alembic check clean	lint, type-check, production bundle, migrations match the models
Backup / restore	executed	pg_dump + data volume -> empty database -> restore -> login and question answered

Not verifiable here (Appendix A): live OpenAI/Gemini calls, a Qdrant *server*, the Docker build and Compose run, GitHub Actions, RAGAS, a cross-encoder model, LangSmith, delivery of spans to an OTLP collector.

2. Business Problem and Objectives

- Collections reach hundreds or thousands of pages; manual review is slow, and keyword search alone cannot count, compare or reason across files.
- Semantic search misses exact names and IDs; spreadsheets need calculation, not similarity; contracts and policies change between versions and teams need to see exactly what changed.
- Answers are only useful when verifiable; unverifiable answers are a compliance and trust risk.
- Enterprises need identity, roles, tenant isolation, document-level sharing, audit, retention and deletion before adopting AI search on confidential material.

Objectives

- **BO-1** Cited answers over large mixed-format collections.
- **BO-2** Exact search with complete counts and page/row locations.
- **BO-3** Reliable calculations on tables (pandas, read-only SQL, charts, trends, anomalies).
- **BO-4** Document intelligence: summaries, comparison and change detection, contradictions, classification, timelines, entities, tables, multilingual (incl. Hindi) retrieval.
- **BO-5** Secure multi-tenant service: JWT + refresh tokens, roles, document ACL, audit, rate limits, retention and deletion.
- **BO-6** Scalable operation: background ingestion, PostgreSQL, Redis, Qdrant, scheduler, metrics and tracing.
- **BO-7** Measurable quality: Recall@K, Precision@K, MRR, citation accuracy, agent success, failed tool calls, latency, tokens, cost.
- **BO-8** Reports people can share (PDF/DOCX/XLSX) with sources and timestamps.

3. Stakeholders and Roles

Role	Needs	Platform permissions
User	Ask, search, summarise, compare, analyse data, generate reports, manage own memory	ask, read, search, summarize, compare, analytics, web, report, memory
Manager	Curate sources, share documents, run evaluations	user + ingest (upload, URL, re-index, versions), share, delete own, evaluate
Administrator	Users and roles, audit, organisation policy (retention, web allow-list, redaction), erase data	manager + manage users, audit, settings, purge, delete any
Operator	Deploy, monitor, back up, upgrade	environment configuration, CLI, worker, scheduler, /metrics
Security / compliance	Isolation, audit, retention, injection and SSRF defences	review this BRD, audit log, security tests

4. Scope

4.1 In scope

- Ingestion of PDF, DOCX, TXT, MD, CSV, XLSX, XLS, JSON, HTML files, URLs and image files (OCR); duplicates, versions, re-index, delete.
- Advanced RAG with Self-RAG option; supervisor with Exact, Document, Data Analyst, Summary, Comparison, Web and Verification agents; parallel execution.
- Reports (PDF/DOCX/XLSX); user-controlled memory; roles, refresh tokens, document sharing; retention; metrics, tracing; React UI; evaluation harness; operations documentation.

4.2 Out of scope / not claimed

- Execution of code or SQL found in documents or prompts (permanently excluded).
- SSO / OIDC / SAML, MFA, SCIM; compliance certifications.

- Reading values out of chart images; understanding images embedded inside PDFs; relation-type extraction for the knowledge graph (edges are co-mentions).
- Neo4j or another graph database; native object storage (a shared volume is used); Celery (the built-in Redis queue is used - decision D1); PaddleOCR (Tesseract is used - D3).
- Correct PDF layout of Hindi/Arabic text in generated PDF reports (use DOCX/XLSX).

5. Input Formats

Every extracted unit carries a location so citations, filters and exact-search results point to the source. Content is validated before parsing (extension allow-list, magic bytes, zip-bomb and macro checks, size and page limits).

Format	Handling	Evidence preserved	Status
PDF	PyMuPDF (unclipped text so off-page glyphs are kept) with pypdf/pdfplumber fallback; bookmarks or font-size headings as sections; ruled tables; OCR for scanned pages; document properties.	file, version, page, section	**Implemented, tested** 1,000 pages tested
CSV / XLSX / XLS	Typed pandas tables for analysis, row units for search; header detection; encodings sniffed.	file, sheet, row, column names	**Implemented, tested**
DOCX	Paragraphs and tables in order, headings as sections, macros rejected, properties.	paragraph / table + row, section	**Implemented, tested**
HTML files, URLs	Readable text and tables; SSRF-guarded fetch with pinned IP and SNI.	URL or paragraph, title, section	**Implemented, tested**
TXT / MD / JSON	Paragraphs / headings / flattened JSON paths; BOM, UTF-8, charset detection, cp1252 fallback.	line, section, JSON path	**Implemented, tested**
Images	Tesseract OCR; optional vision caption.	'image (OCR)' / 'image caption'	**Implemented, partly tested** captions: scripted model only

Identity: the sha256 of the file decides. An identical file is refused (existing document returned, unless explicitly allowed); the same name with new content becomes version N+1; older versions are kept for comparison and rollback but are not searched.

6. Functional Requirements

Ingestion

ID	Requirement	Description	Status
FR-01	Formats	PDF, DOCX, TXT, MD, CSV, XLSX, XLS, JSON, HTML, URLs, scanned PDF and PNG/JPG with OCR.	**Implemented, tested**
FR-02	Large documents	1,000+ page PDFs ingest in the background with progress; counts stay exact.	**Implemented, tested**
FR-03	Duplicates and versions	Content hash duplicate refusal; versions; make-current rollback; change detection between versions.	**Implemented, tested**
FR-04	Resilience	Corrupt, truncated, empty, binary, wrong-extension files fail cleanly with a reason; nothing partial is stored.	**Implemented, tested**
FR-05	Metadata	Page, sheet, row, paragraph, section, URL, properties, language, classification, content hash.	**Implemented, tested**
FR-06	Delete and re-index	Delete removes text, chunks, vectors, original; optional all versions; re-index in place.	**Implemented, tested**

Document intelligence

ID	Requirement	Description	Status
FR-10	Exact search	Full-corpus literal search: counts per document, pages, excerpts, pagination; documented matching rules; CJK/Thai substring rules.	**Implemented, tested**
FR-11	Summaries	Short, detailed, executive, chapter-by-chapter and named-section summaries over the whole document.	**Implemented, tested**
FR-12	Comparison and change detection	Sentence alignment with number changes between documents and between versions.	**Implemented, tested**
FR-13	Contradiction detection	Numeric / negation / antonym candidates across documents; LLM confirmation when configured.	**Implemented, partly tested**
FR-14	Entities, classification, timeline, tables	Heuristic entities (+ verbatim-validated LLM), closed-taxonomy classification, chronological events with citations, structured table extraction.	**Implemented, tested**
FR-15	Cross-document QA	Evidence from every relevant document is included; no multi-hop reasoning.	**Implemented, partly tested**
FR-16	Multilingual / Hindi	Unicode tokenisation, Hindi stop-words and routing keywords, Hindi table questions, Hindi summaries; PDF reports do not shape Devanagari.	**Implemented, partly tested**
FR-17	Reports	Summary, executive, comparison, research, analytics, exact-search and evaluation reports as PDF/DOCX/XLSX with sources and timestamps.	**Implemented, tested**

Retrieval

ID	Requirement	Description	Status
FR-20	Hybrid retrieval	Semantic, BM25, hybrid (RRF); rewrite; multi-query; HyDE (never evidence); metadata filters; parent-child; compression; reranking; corrective loop.	**Implemented, tested**
FR-21	Self-RAG	Retrieve-or-not, passage relevance, sentence-level support critique, bounded regeneration, removal of unsupported statements.	**Implemented, tested**
FR-22	Cross-encoder reranking	Optional local model; code path only.	**Implemented, partly tested**
FR-23	Vector stores	Qdrant (production) and ChromaDB (development); Qdrant tested in embedded mode only.	**Implemented, partly tested**
FR-24	Graph RAG	Entity co-mention graph, neighbour/path queries, retrieval expansion.	**Implemented, tested**

Agents

ID	Requirement	Description	Status
FR-30	Supervisor	Rules or LLM routing; compound questions fan out in parallel (LangGraph Send) and are merged.	**Implemented, tested**

ID	Requirement	Description	Status
FR-31	Data Analyst	Pandas operations, profile, duplicates, IQR/z-score anomalies, monthly/quarterly/yearly trends with YoY, top-N, charts; read-only SQL sandbox; NL-to-SQL validated by the sandbox.	**Implemented, tested**
FR-32	Comparison Agent	Documents, versions, topics, contradictions.	**Implemented, tested**
FR-33	Web Research	Per-tenant approved domains, redirect check, separate web evidence (origin=web); a real search provider was not called.	**Implemented, partly tested**
FR-34	Verification	Number grounding, citation validation against stored text (page/row/excerpt), judge, widened-retrieval retry, strict fallback.	**Implemented, tested**
FR-35	Reliability	Typed state, checkpoints, retry limits, per-node timeouts, transient-error retries, failure isolation per branch.	**Implemented, tested**

Memory

ID	Requirement	Description	Status
FR-40	Conversation memory	Window + rolling summary, per-user isolation.	**Implemented, tested**
FR-41	User-controlled memory	Opt-in notes recalled as context (never evidence), incognito, private conversations, reset, delete, sensitive-data refusal, optional redaction.	**Implemented, tested**

Security and governance

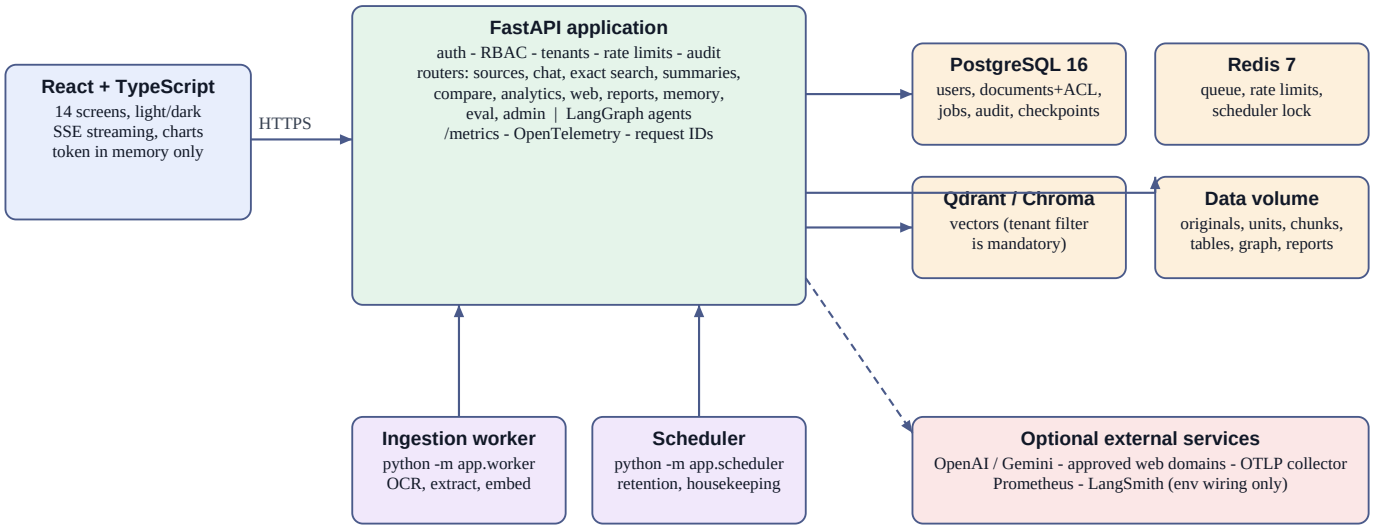
ID	Requirement	Description	Status
FR-50	Authentication	JWT access token (15 min) + rotating refresh token in an httpOnly cookie with reuse detection.	**Implemented, tested**
FR-51	Authorisation	Roles user / manager / admin; tenant and workspace isolation; document-level sharing (organisation / private / users and roles).	**Implemented, tested**
FR-52	Protection	Upload validation, rate limits, audit trail, prompt-injection defences, SSRF protection, SQL authorizer, spreadsheet formula neutralisation.	**Implemented, tested**
FR-53	Retention and deletion	Per-tenant retention enforced by a scheduler (dry run available); erase document, user, memory, tenant.	**Implemented, tested**
FR-54	SSO / MFA	Not implemented.	**Pending**

Platform

ID	Requirement	Description	Status
FR-60	Database	PostgreSQL with Alembic migrations and indexes; SQLite for development.	**Implemented, tested**
FR-61	Background processing	Redis-backed queue with status, retries, progress; scheduled tasks.	**Implemented, tested**
FR-62	API	Groups: auth, users/roles, documents, jobs, chat + streaming, exact search, summaries, comparison, analytics, web, reports, memory, evaluation, admin/audit; pagination; OpenAPI (75 operations).	**Implemented, tested**
FR-63	User interface	14 screens, light/dark, streaming, charts, filters, citation navigation with PDF page preview.	**Implemented, tested**
FR-64	Observability	Prometheus metrics, OpenTelemetry spans; LangSmith via environment variables (not verified live).	**Implemented, partly tested**
FR-65	Deployment	Dockerfile, Compose, CI definitions, backup and restore scripts; image/Compose/CI not executed.	**Implemented, partly tested**

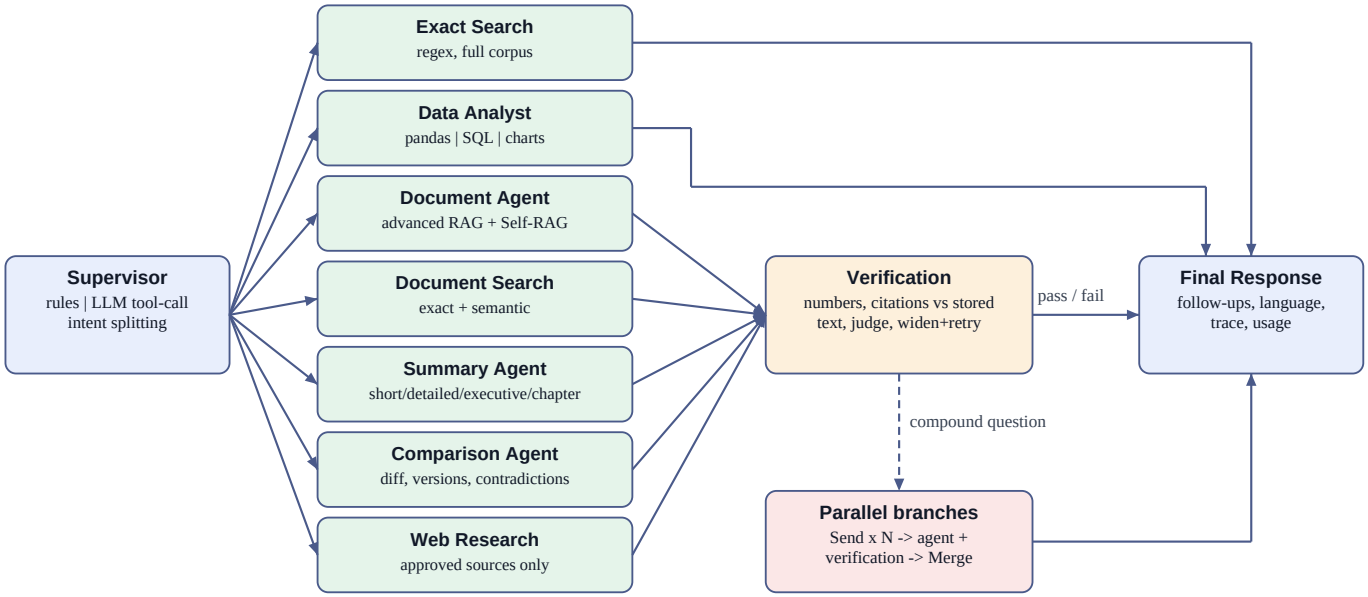
7. Architecture

System context - API tier is stateless; state lives in PostgreSQL, Redis, the vector store and the data volume



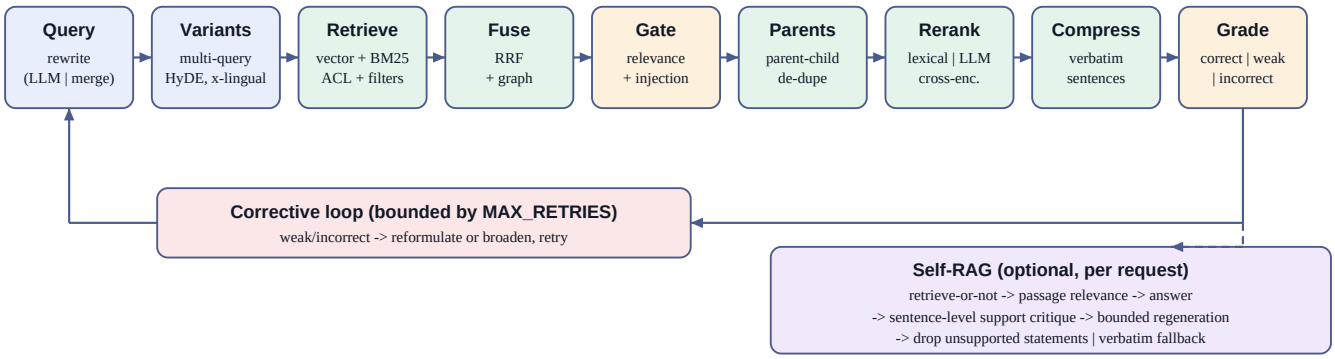
Agent graph. The supervisor routes each question; compound questions ("count X, and what is the highest Y?") fan out to parallel branches that each run an agent and a verification pass, then merge into one sectioned answer. Generative agents pass through Verification; exact and data answers are deterministic and have their citations validated in the final step.

LangGraph StateGraph - typed, checkpointed state; every node has timeout, retry, metrics and a trace span



semantic -> Web when abstained and fallback is on; data -> Document Agent when the question is not about the tables; verify retry: 1st widened retrieval, then strict extraction

Retrieval pipeline. Every step is configurable per request; LLM-dependent steps degrade to deterministic fallbacks.



Data tier. PostgreSQL holds identities, tenants, one row per document *version* (hash, visibility, metadata), document permissions, jobs, conversations, memory notes, reports, audit events and LangGraph checkpoints. Content (originals, units, chunks, tables, graphs, reports) lives on the data volume; vectors in Qdrant/Chroma/local. Migrations: 0001 baseline, 0002 registry/versions/ACL/roles/refresh tokens/memory/reports; `alembic check` proves they match the models.

Key decisions and trade-offs

Decision	Rationale	Trade-off
Keep the built-in Redis queue instead of Celery (D1)	Tested idempotent claim, bounded retries, crash re-delivery; same capability	No Celery ecosystem tooling (Flower, beat); scheduler is a small dedicated process
PyMuPDF with pypdf fallback (D2)	10-20x faster, headings, page images; unclipped text avoids dropped glyphs	AGPL-3.0 / commercial licence - operators must decide; pinned below 2.0
Tesseract, not PaddleOCR (D3)	No large model downloads	Lower accuracy on difficult scans
NetworkX graph (D6)	No extra service; independently testable	Not for very large graphs; edges are co-mentions
SQL via SQLite authorizer sandbox (D7)	Real SQL semantics, whitelisted operations, time/row limits	SQLite dialect; spreadsheets are copied in memory per request (cached parse)
Citation validation against stored text	A citation must exist and contain its excerpt	Spreadsheet cells are re-formatted, so rows use a looser coverage threshold
Heuristic contradictions + optional LLM judge	Works offline; candidates are labelled as such	False positives/negatives without a model

8. Security, Privacy and Data Lifecycle

Layer	Controls (all exercised by tests unless noted)
Identity	bcrypt; 15-min JWT (memory only in the browser); rotating 14-day refresh token (httpOnly, SameSite=Strict, hashed in DB, family revoked on reuse); roles user/manager/admin; legacy roles migrated.
Isolation	tenant + workspace in every storage address; mandatory Qdrant filters; document ACL (organisation / private / restricted to users or roles); hidden = missing.
Input	size, magic bytes, zip-bomb, macros; SSRF-safe fetch with pinned IP + SNI; prompt-injection scan, data fences, closed tool registry.
Execution	no generated code executed; SQL = one SELECT under an authorizer (function/table whitelist, no PRAGMA/ATTACH/DDL/DML/recursion), length/time/row limits.
Output	output guard; citation validation; XLSX formulas neutralised; CSP (self, data:, blob: images); no secrets in logs or audit.
Data lifecycle	per-tenant retention (documents, conversations, audit >= 30 days, reports) by scheduler with dry run; erase user / tenant / memory; optional masking of e-mail, phone, card (Luhn), ID numbers and secrets in stored messages (pattern-based, not a guarantee).
Operations	Redis-shared rate limits; append-only audit (hashes, never text); secrets from env or *_FILE (no vault integration); production start-up validator.

Residual risks. Heuristic injection detection can be bypassed; PDF/OCR extraction can be wrong; PyMuPDF licence; a shared volume is a single point of failure and a scaling limit; no SSO/MFA; no penetration test was performed; masking is pattern-based.

9. Evaluation and Performance (measured)

Synthetic corpus (42 questions over PDF, DOCX, XLSX, CSV, MD, TXT, JSON; offline hash embedder, no LLM). The set is small and synthetic: it detects regressions and compares strategies; it is **not** a real-world accuracy benchmark. Hard paraphrases fail with the offline embedder and are listed in the report.

Metric	Value	Metric	Value
Recall@5	90.9%	Route accuracy	100.0%
Precision@5	18.2%	Exact-count accuracy	100.0%
MRR	0.909	Spreadsheet-value accuracy	100.0%
nDCG@5	0.909	Abstained / flagged unanswerable	100.0%
Citation accuracy	100.0%	Faithfulness (lexical proxy)	88.9%
Agent execution success	100.0%	Failed tool calls	0 of 49
Latency p50 / p95	18 / 28 ms	Tokens / cost per query	0 (offline)

Extended corpus (14 questions): versions, comparison, contradictions, Hindi (exact, retrieval, table, summary), analytics (trend, outliers), parallel agents, abstention. Route accuracy 100.0%, keyword accuracy 100.0%, exact counts 100.0%, data values 100.0%, required sources cited 100.0%, stale (superseded-version) content absent 100.0%, citation accuracy 100.0%, agent success 100.0%, failed tool calls 0. Faithfulness and relevancy shown by the harness are lexical/embedding *proxies*; LLM-judge values and RAGAS need a configured model and the ragas package (not measured here).

Performance on this machine (4 vCPU, offline configuration; see docs/PERFORMANCE.md)

Workload	Result
1,000-page PDF (0.96 MB, 5,000 chunks)	ingest 5.6 s (177.3 pages/s) - exact search 106 ms with the correct count (25) - complete /ask p50 86 ms
200,000-row CSV (6.1 MB)	ingest 18.2 s - tables loaded in 2929 ms (then 1.2 ms cached) - profile 1255 ms - monthly trend 1285 ms - SQL GROUP BY 65 ms
Reports	summary of the whole 1,000-page document built in 1.6 s; rendering PDF 0.14 s, DOCX 0.06 s, XLSX 0.08 s
Peak memory	705 MB for the whole benchmark process

10. User Interface and API

Screen	Purpose
Sign-in / registration	JWT login; silent refresh from the httpOnly cookie after a reload
Dashboard	document and job counts, library mix by type and class, problems needing attention, quick actions
Library + upload + ingestion progress	drag-and-drop, URLs, per-file progress, duplicate notice with 'upload anyway', search/filter/paginate, versions and what-changed, sharing, class, entities, timeline, re-index, delete
Chat	streaming answers, confidence, citation validity, citations with 'view in context', filters, Self-RAG and incognito toggles, charts, follow-ups, voice
Citation preview	PDF page image with the cited text highlighted and page navigation; passage with neighbours for other formats; web citations marked
Exact search	term, case/whole-word, document filter; totals per document, clickable page numbers, paginated matches
Comparison	two documents, versions of one document, contradiction scan
Data analyst	ask in words, profile, outliers, trends with YoY, read-only SQL with schema, charts with PNG download
Reports	choose type/format/documents, generate, list, download, delete
History	search, read, rename, make private, delete conversations
Users and roles / Admin	create, change role, disable, delete; audit log; platform settings; erase tenant data
Settings	theme (system/light/dark), memory preferences and notes, organisation policy (retention, web allow-list, redaction), retention sweep preview/run
Evaluation	run sample or extended evaluation, strategy comparison, export PDF/DOCX/XLSX

API groups (75 operations, pagination via limit/offset and X-Total-Count, OpenAPI at /openapi.json and docs/openapi.json): auth, sources (upload, URL, versions, sharing, previews, timeline, classification), jobs, chat (ask, stream, conversations), search/exact, summaries, compare, analytics (tables, profile, anomalies, trend, sql, ask, chart), web/research, reports, memory, eval, admin (users, audit, settings, tenant-settings, maintenance, purge), health, metrics.

11. Risks, Limitations and Trade-offs

Risk / limitation	Impact	Mitigation / status
Offline hash embedder is lexical	paraphrase queries miss	configure a real embedder; hard cases are listed in the evaluation report
Heuristic contradictions	false positives/negatives	labelled 'candidate'; LLM judge when configured; manual review
PyMuPDF is AGPL/commercial	licence obligations	MKA_PDF_ENGINE=pypdf avoids it (slower, no page images)
PDF tables only where ruled lines exist	borderless tables not structured	text remains searchable
Hindi in PDF reports not shaped	unreadable conjuncts in PDF	use DOCX/XLSX; text is Unicode
Shared data volume	scaling and single point of failure	network filesystem now; object storage is future work
Node timeouts cannot kill threads	timed-out work finishes in the background	client timeouts on providers; result discarded
No SSO/MFA; no penetration test	enterprise onboarding gaps	planned; hardening documented
Docker/Compose/CI not executed here	first run may need fixes	files validated statically; process-level smoke test executed
Single-node tested only	multi-replica behaviour	advisory locks and Redis lock designed for replicas; not load tested

12. Acceptance Criteria

ID	Criterion	Status	Evidence
AC-01	A 1,000+ page PDF ingests and an exact name search returns the exact count with pages	**Implemented, tested**	test_large_documents.py
AC-02	Max sales from an Excel sheet is computed with row citations	**Implemented, tested**	test_tabular.py
AC-03	Semantic question answered with validated citations; missing evidence yields 'could not verify'	**Implemented, tested**	test_agents.py, test_verification_selfrag.py
AC-04	Two documents / two versions compared; changes with numbers reported	**Implemented, tested**	test_compare.py
AC-05	PDF tables, OCR, DOCX, XLSX, XLS, HTML handled; corrupted files fail cleanly	**Implemented, tested**	test_ingestion_v3.py, test_phase2_ingestion.py
AC-06	Tenant isolation and unauthorised access are enforced	**Implemented, tested**	test_auth_tenancy.py, test_foundation.py
AC-07	Agent failures and timeouts degrade to honest answers; branches isolated	**Implemented, tested**	test_operations.py
AC-08	Reports are generated as PDF, DOCX and XLSX with sources and timestamps	**Implemented, tested**	test_reports_memory.py
AC-09	Retention sweeps and erasure work per tenant	**Implemented, tested**	test_operations.py, test_reports_memory.py
AC-10	Migrations apply to an empty PostgreSQL and match the models; backup/restore works	**Implemented, tested**	test_foundation.py, scripts/backup.sh
AC-11	Full UI flow works in a real browser	**Implemented, tested**	frontend/e2e/smoke.mjs
AC-12	Quality measured: citation accuracy, agent success, failed tool calls, latency	**Implemented, tested**	test_evaluation.py
AC-13	Live provider behaviour (OpenAI/Gemini), RAGAS, cross-encoder, Qdrant server	**Pending**	not executed - needs credentials / downloads / services
AC-14	Docker build, Compose stack, GitHub Actions run	**Pending**	not executed - no container runtime or CI available
AC-15	Accuracy on a real customer corpus with real models	**Pending**	pending

Appendix A - What could not be verified, and why

- Live OpenAI / Gemini: no credentials or network egress to the providers. All LLM-dependent behaviour was tested with scripted test doubles (orchestration, validation, fallbacks), not model quality.
- Qdrant server: starting the binary was not permitted; the embedded client was tested. Hugging Face is unreachable, so no cross-encoder or multilingual embedding model was run.
- Docker / Compose / GitHub Actions: no daemon or runner. The workflow, Dockerfile and compose file are written and statically validated only.
- RAGAS and LangSmith: packages / network unavailable; the RAGAS adapter was tested with a stub module and LangSmith is configured through environment variables only.
- OTLP delivery: spans were verified with an in-memory exporter, not against a collector.
- Mobile layouts: CSS breakpoints exist but the UI was only exercised at desktop width.

Appendix B - Documents shipped with the source

- README.md (quick start, usage, status) - docs/ARCHITECTURE.md (diagrams) - docs/OPERATIONS.md (deployment, monitoring, backup/recovery, retention, upgrades)
- docs/PERFORMANCE.md and docs/perf.json - docs/REQUIREMENTS_TRACE.md - docs/API.md and docs/openapi.json - docs/eval/ (sample and extended evaluation reports in JSON, Markdown, HTML, PDF, DOCX, XLSX) - docs/PLAN_v3.md